

Informe tecnico de la Version 2: arquitectura MVC con calculo concurrente

Maria Jose Betancourt - A00403987

Joshua Sayur - A00404707

Martin Borrero Herrera - A00403871

Javier Andres Paz Luna - A00405619

Camilo Andres Martinez Moreno - A00405205

Universidad Icesi

Ingenieria de Software IV

Proyecto final

Informe tecnico de la Version 2: arquitectura MVC con calculo concurrente

Objetivo

El objetivo del proyecto es calcular la velocidad promedio por ruta y por mes a partir de registros de posicion GPS de buses del SITM-MIO de Cali. La Version 2 conserva la arquitectura monolitica organizada bajo el patron MVC, pero incorpora concurrencia en la capa de servicio para reducir el tiempo de calculo sin modificar la salida funcional del programa.

La version concurrente mantiene los mismos datos de entrada, la misma formula de distancia, el mismo criterio de descarte de outliers y la misma exportacion a Excel. Por tanto, el cambio principal no es funcional sino de rendimiento y organizacion del procesamiento interno.

Metodo de calculo

Datos de entrada

El programa utiliza dos archivos CSV. El archivo datagrams-MiniPilot.csv, o su equivalente de mayor tamano, contiene los registros GPS de los buses. El archivo lines-241-ActiveGT.csv contiene las rutas activas que deben ser consideradas durante el calculo.

Tabla 1

Columnas utilizadas del archivo de datagrams

Indice	Campo	Descripcion
[4]	latitude	Latitud GPS almacenada como entero; se divide entre 1e7 para obtener grados decimales.
[5]	longitude	Longitud GPS almacenada como entero; se divide entre 1e7 para obtener grados decimales.

[7]	lineId	Identificador de la ruta asignada al bus en el sistema IVU.
[10]	datagramDate	Fecha y hora del evento con formato yyyy-MM-dd HH:mm:ss.
[11]	busId	Identificador del bus en el sistema IVU.

Nota. La tabla resume las columnas leídas por DatagramRepository para construir los objetos Datagram.

Algoritmo de procesamiento

Primero, las rutas activas se cargan en un HashSet para permitir búsquedas en tiempo constante. Luego, los datagrams se recorren una sola vez para descartar rutas no activas y agrupar los registros validos por la llave busId|lineId. Esta agrupacion es importante porque solo deben compararse posiciones consecutivas de un mismo bus dentro de una misma ruta.

Despues del agrupamiento, el servicio define la cantidad de workers segun los procesadores disponibles mediante Runtime.getRuntime().availableProcessors(). Ese valor se limita para evitar crear menos de un hilo o mas hilos que grupos de trabajo existentes. Los grupos se reparten en particiones mediante una estrategia round-robin y cada particion se ejecuta como una tarea independiente en un ExecutorService.

Cada worker ordena sus grupos por fecha, calcula la distancia entre pares consecutivos mediante la formula Haversine y calcula la velocidad como distancia dividida entre tiempo. Se descartan pares con tiempo menor o igual a cero, distancia menor o igual a cero o velocidad superior a 80 km/h. Las velocidades validas se acumulan por lineId y mes usando DoubleSummaryStatistics. Al final, el hilo principal espera los Future, fusiona las estadisticas parciales y genera los objetos SpeedRecord.

Formula Haversine

La distancia entre dos puntos geograficos se calcula con la formula Haversine:

$$a = \sin^2(\Delta_{\text{lat}}/2) + \cos(\text{lat}_1) * \cos(\text{lat}_2) * \sin^2(\Delta_{\text{lon}}/2)$$

$$\text{distancia} = 6371 * 2 * \text{atan2}(\sqrt{a}, \sqrt{1-a}) \text{ [km]}$$

En esta formula, las latitudes y longitudes se expresan en radianes. El valor 6371 corresponde al radio aproximado de la Tierra en kilometros.

Optimizaciones aplicadas

La Version 2 reutiliza las optimizaciones de V1 y agrega concurrencia controlada en la etapa de calculo. La estrategia evita compartir estructuras mutables durante el procesamiento intensivo, lo que reduce el riesgo de condiciones de carrera.

Tabla 2

Optimizaciones implementadas en la Version 2

Optimizacion	Descripcion
Agrupacion por bus y ruta	Agrupar los datagrams por busId lineId antes de calcular velocidades para evitar comparaciones incorrectas.
ExecutorService	Ejecuta particiones independientes en un pool fijo de hilos.
Cantidad de hilos adaptable	Usa los procesadores disponibles y limita el valor al numero real de grupos.
Particionado round-robin	Distribuye los grupos de manera simple entre workers.
Estadisticas parciales	Cada worker acumula resultados en su propio HashMap de DoubleSummaryStatistics.

Fusion posterior	El hilo principal combina las estadísticas cuando las tareas terminan.
Ordenamiento localizado	Ordena cada grupo por fecha en lugar de ordenar una lista global completa.
Compatibilidad con Java 17	Usa Math.max y Math.min para limitar workers, evitando APIs no disponibles en Java 17.

Nota. Las optimizaciones buscan mejorar la etapa de calculo sin alterar el contrato de entrada y salida del sistema.

Resultados

Los tiempos fueron medidos en la maquina local usando datagrams-MiniPilot.csv y lines-241-ActiveGT.csv. La comparacion se centra en la etapa de calculo, porque la carga del CSV continua siendo secuencial y depende fuertemente del disco y del estado del sistema.

Tabla 3

Comparacion de tiempos entre V1 y V2

Version	Carga (ms)	Calculo (ms)	Total (ms)	Registros	Salida
V1 secuencial	7774	4474	13045	97	output/resultados-v1.xlsx
V2 concurrente	8190	1651	11128	97	output/resultados-v2.xlsx

Nota. Los tiempos pueden variar entre ejecuciones por cache de disco, carga del sistema o configuracion del IDE.

La etapa de calculo paso de 4474 ms en V1 a 1651 ms en V2. Esto equivale a una aceleracion aproximada de 2.71 veces y a una reduccion cercana al 63.1% en el tiempo de calculo. El tiempo total disminuyo de 13045 ms a 11128 ms; esta mejora total es menor porque la lectura del CSV sigue siendo secuencial.

Arquitectura del sistema

La solución conserva una arquitectura monolítica con separación de responsabilidades mediante MVC. Esta separación facilita explicar, probar y modificar el sistema sin mezclar lectura de datos, lógica de negocio, exportación y coordinación del flujo principal.

Tabla 4

Responsabilidades por capa

Capa	Clases	Responsabilidad
Modelo	Datagram, Route, SpeedRecord	Representan las entidades del dominio.
Repositorio	DatagramRepository, RouteRepository	Leen los archivos CSV y construyen objetos del dominio.
Servicio	SpeedCalculationService	Filtra, agrupa, calcula velocidades, ejecuta tareas concurrentes y consolida estadísticas.
Vista	ExcelView	Exporta los resultados a un archivo Excel.
Controlador	ProcessingController	Coordina repositorios, servicio y vista; también mide tiempos.
Aplicación	Main	Define las rutas de entrada y salida y ejecuta el proceso.

Nota. La concurrencia se concentra en la capa de servicio para mantener aislada la lógica de rendimiento.

Validación

La implementación fue validada con la suite de pruebas automatizadas de Gradle y con una ejecución completa del punto de entrada Main. La ejecución generó el archivo output/resultados-v2.xlsx con 97 registros.

Comandos de validación ejecutados:

```
.\gradlew.bat test
```

```
.\gradlew.bat run
```

Conclusiones

La Version 2 mantiene la misma salida funcional de la Version 1, pero reduce el tiempo de calculo al dividir el procesamiento en grupos independientes de bus y ruta. La mejora se observa principalmente en la etapa de calculo, mientras que la lectura del archivo CSV permanece como un componente secuencial relevante dentro del tiempo total.

La implementacion concurrente evita condiciones de carrera porque cada worker opera sobre sus propias particiones y estadisticas parciales. La fusion de resultados ocurre posteriormente en el hilo principal, lo que mantiene el diseno simple y verificable.

Referencias

No se utilizaron fuentes externas en la elaboracion de este informe. La descripcion corresponde al codigo fuente y a las mediciones locales del proyecto.